

3 XML (eXtensible Markup Language)

- Also see
 - Chapter 15 of text
 - <https://www.w3schools.com/xml>

3.1 Introduction/Basics

- Widely supported open (i.e., non-proprietary) technology for describing data that has become the standard format for data exchanged between applications over the Internet
- Permits document authors to create markup for virtually any type of information
- Can create entirely new markup languages that describe specific types of data, including mathematical formulas, chemical molecular structures, music and recipes
- Describes data in a way that human beings can understand and computers can process

3.2 XML Basics

```
1  <?xml version = "1.0"?>
2
3  <!-- Baseball player structured with XML -->
4  <player>
5      <firstName>John</firstName>
6      <lastName>Doe</lastName>
7      <battingAverage>0.375</battingAverage>
8  </player>
```

```
1  <?xml version = "1.0"?>
2
3  <!-- article.xml -->
4  <!-- Article structured with XML -->
5  <article>
6      <title>Simple XML</title>
7      <date>July 4, 2007</date>
8      <author>
9          <firstName>John</firstName>
10         <lastName>Doe</lastName>
11     </author>
12     <summary>XML is pretty easy.</summary>
13     <content>This chapter presents examples that use XML.</content>
14 </article>
```

3.2.1 XML documents

- contain text that

- represents content (i.e., data), such as John (line 5 above), and
 - elements that specify the document's structure, such as firstName (line 5 above)
- delimit elements with start tags and end tags
 - Start tag
 - Consists of the element name in angle brackets (e.g., <player> and <firstName>)
 - End tag
 - Consists of the element name preceded by a forward slash (/) in angle brackets (e.g., </firstName> and </player>)
 - Empty XML elements
 - Have no content
 - Indicated like this
 - <element></element>
 - or you can use a self-closing tag
 - <element />
- Every XML document must have exactly one root element that contains all the other elements

3.2.2 XML Vocabularies (i.e., XML-based markup languages)

- Provide a means for describing particular types of data in standardized, structured ways.
- Examples: XHTML (Extensible HyperText Markup Language), MathML™ (for mathematics), VoiceXML™ (for speech), CML (Chemical Markup Language – for chemistry), XBRL (Extensible Business Reporting Language – for financial data exchange), others
- Much of the massive amounts of data that are currently stored on the Internet in various formats (databases, web pages, text files, etc) are transformed into XML for the purpose of transferring them between systems
- Industry groups are developing custom XML vocabularies for most major industries that will allow business applications to communicate in common languages, e.g., using web services

3.2.3 Viewing and Modifying XML Documents

- An XML document is a text file that usually ends with the .xml filename extension
- Does not require special software to view
- Any text editor that supports ASCII/Unicode characters can open XML documents for viewing and editing

- Also, most web browsers can display XML documents in a formatted manner that shows the XML's structure

3.2.4 Processing XML Documents

- Requires software called an XML parser (or XML processor)
- A parser makes the document's data available to applications
- XML parser: Identifies components of XML documents and stores them in a data structure for manipulation
- While reading an XML document's contents, a parser checks that the document follows the syntax rules specified by the W3C's XML Recommendation (www.w3.org/XML)
 - Case sensitivity
 - Single root element, a start tag and end tag for each element, and properly nested tags
- Well-formed XML document
 - A document that conforms to the syntax rules
 - Parsers can provide access to XML-encoded data in well-formed documents only

3.2.5 Formatting and Manipulating XML Documents

- Most XML documents contain only data, so applications that process XML documents must decide how to manipulate or display the data
 - XSLT (eXtensible Stylesheet Language Transformations)
 - The recommended style sheet language for XML
 - Example below uses XSLT to transform the XML document into HTML, before it is displayed in a browser

```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <cd>
    <title>Empire Burlesque</title>
    <artist>Bob Dylan</artist>
    <country>USA</country>
    <company>Columbia</company>
    <price>10.90</price>
    <year>1985</year>
  </cd>
  <cd>
    <title>Hide your heart</title>
    <artist>Bonnie Tyler</artist>
    <country>UK</country>
    <company>CBS Records</company>
    <price>9.90</price>
    <year>1988</year>
  </cd>
  <cd>
    <title>Greatest Hits</title>
    <artist>Dolly Parton</artist>
    <country>USA</country>
    <company>RCA</company>
    <price>9.90</price>
    <year>1982</year>
  </cd>
</catalog>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/
XSL/Transform">
<xsl:template match="/">
<html>
<body>
  <h2>My CD Collection</h2>
  <table border="1">
    <tr bgcolor="#9acd32">
      <th style="text-align:left">Title</th>
      <th style="text-align:left">Artist</th>
    </tr>
    <xsl:for-each select="catalog/cd">
      <tr>
        <td><xsl:value-of select="title"/></td>
        <td><xsl:value-of select="artist"/></td>
      </tr>
    </xsl:for-each>
  </table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan
Hide your heart	Bonnie Tyler
Greatest Hits	Dolly Parton

- XPath (XML Path Language): A language for accessing parts of an XML document
 - Uses path expressions to select nodes or node-sets in an XML document
- XQuery: The language for querying XML data
 - XQuery for XML is like SQL for databases

3.3 Structuring Data

```
1  <?xml version = "1.0"?>
2
3  <!-- article.xml -->
4  <!-- Article structured with XML -->
5  <article>
6      <title>Simple XML</title>
7      <date>July 4, 2007</date>
8      <author>
9          <firstName>John</firstName>
10         <lastName>Doe</lastName>
11     </author>
12     <summary>XML is pretty easy.</summary>
13     <content>This chapter presents examples that use XML.</content>
14 </article>
```

3.3.1 XML document

- Begins with an optional XML declaration
 - Declaration identifies the document as an XML document, includes version attribute
- Blank Space and Comments
 - Blank lines (normally ignored by XML parsers), white spaces and indentation help improve readability
 - XML comments begin with <!-- and end with -->
- Root Node and Prolog
 - Root element: Lines 5 – 14
 - Prolog
 - The lines that precede the root element (lines 1 – 4)
 - The XML declaration must appear before the comments and any other markup
- XML Element Names
 - (Invent and) use element names to mark up your data
 - XML element names can be of any length and may contain letters, digits, underscores, hyphens and periods
 - But must begin with either a letter or an underscore, and they should not begin with “xml” in any combination of uppercase and lowercase letters
- Nesting XML Elements
 - XML elements are nested to form hierarchies, with the root element at the top of the hierarchy

- Nesting enables parent/child relationships between data items
 - Example: Elements title, date, author, summary and content are children of article
- Any element that contains other elements is a container (or parent) element
- Elements nested inside a container element are child elements (or children) of that container element
- A node can have multiple children, but only one parent
- Nodes with the same parent node are referred to as siblings

3.3.2 Viewing an XML Document in a Web Browser

- XML is a technology for describing the structure (and not formatting) of data
- Formatting and displaying of XML data are application-specific issues
- Example
 - Display of XML above using the Mozilla and Chrome browsers are shown below
 - Each browser has a built-in style sheet to format the data
- Each of these browsers provides a mechanism to expand or collapse a container node
 - Collapsing
 - The hyphen (-) in Mozilla, the down arrow in Chrome
 - Expanding
 - The plus (+) in Mozilla, the right arrow in Chrome

Mozilla

```
<!-- article.xml -->
<!-- Article structured with XML -->
-<article>
  <title>Simple XML</title>
  <date>July 4, 2007</date>
  -<author>
    <firstName>John</firstName>
    <lastName>Doe</lastName>
  </author>
  <summary>XML is pretty easy.</summary>
  <content>This chapter presents examples that use XML.</content>
</article>
```

Chrome

```
<!-- article.xml -->
<!-- Article structured with XML -->
▼<article>
  <title>Simple XML</title>
  <date>July 4, 2007</date>
  ▼<author>
    <firstName>John</firstName>
    <lastName>Doe</lastName>
  </author>
  <summary>XML is pretty easy.</summary>
  <content>This chapter presents examples that use XML.</content>
</article>
```

3.3.3 XML Namespace

- Given the following XML documents:

```
<table>
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

```
<table>
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

- Note: Different concepts, but conflict in **table** element name
- **Solving the Name Conflict Using a Prefix**

```
<h:table>
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

- A namespace for the prefix must be defined, using the **xmlns** attribute (see below)

```
<root>

  <h:table xmlns:h="http://www.w3.org/TR/html4/">
    <h:tr>
      <h:td>Apples</h:td>
      <h:td>Bananas</h:td>
    </h:tr>
  </h:table>

  <f:table xmlns:f="https://www.w3schools.com/furniture">
    <f:name>African Coffee Table</f:name>
    <f:width>80</f:width>
    <f:length>120</f:length>
  </f:table>

</root>
```

- Namespaces can also be declared in the XML root element

```
<root xmlns:h="http://www.w3.org/TR/html4/"
xmlns:f="https://www.w3schools.com/furniture">

  <h:table>
    <h:tr>
      <h:td>Apples</h:td>
      <h:td>Bananas</h:td>
    </h:tr>
  </h:table>

  <f:table>
    <f:name>African Coffee Table</f:name>
    <f:width>80</f:width>
    <f:length>120</f:length>
  </f:table>

</root>
```

- Default Namespaces**

- Save us from using prefixes in all the child elements

```
<table xmlns="http://www.w3.org/TR/html4/">
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

```
<table xmlns="https://www.w3schools.com/furniture">
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

3.3.4 Well Formed and Valid XML Documents

- An XML document with correct syntax is called "Well Formed".
 - XML documents must have a root element
 - XML elements must have a closing tag
 - XML tags are case sensitive
 - XML elements must be properly nested
 - XML attribute values must be quoted
- Errors are not allowed in XML documents
- Well-formed XML documents are not necessarily valid
- A valid XML document must be
 - Well formed and
 - Must conform to a document type definition
- Two different XML document type definitions
 - DTD - The original Document Type Definition
 - XML Schema - An XML-based alternative to DTD
- Applications should use DTDs or schemas to confirm whether XML documents are valid (i.e., its elements contain the proper attributes and appear in the proper sequence)

3.3.5 XML DTD

- A DTD can be declared inside the XML file (internal DTD), or as a separate file (external DTD) that can be referred from an XML document
- **Internal DTD**
 - The DTD must be wrapped inside the <!DOCTYPE> definition

XML document with an internal DTD

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend</body>
</note>
```

- **!DOCTYPE note** defines that the root element of this document is note
- **!ELEMENT note** defines that the note element must contain four elements: "to,from,heading,body"
- **!ELEMENT to** defines the to element to be of type "#PCDATA"
- **!ELEMENT from** defines the from element to be of type "#PCDATA"
- **!ELEMENT heading** defines the heading element to be of type "#PCDATA"
- **!ELEMENT body** defines the body element to be of type "#PCDATA"
- **Corresponding External DTD**
 - <!ELEMENT note (to,from,heading,body)>
 - <!ELEMENT to (#PCDATA)>
 - <!ELEMENT from (#PCDATA)>
 - <!ELEMENT heading (#PCDATA)>
 - <!ELEMENT body (#PCDATA)>
- **XML document with a reference to the external DTD**

XML document with a reference to an external DTD

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

3.3.5.1 DTD XML Building Blocks

- Elements
 - Attributes
 - Entities
 - PCDATA
 - CDATA
-
- Elements: Main building blocks, like in HTML
 - Attributes
 - Like in HTML, provide extra information about elements
 - Example:
 - Entities
 - Like in HTML, these are characters with special meanings
 - Entities are expanded when a document is parsed by an XML parser

Entity References	Character
<	<
>	>
&	&
"	"
'	'

- PCDATA (parsed character data)
 - Character data: the text found between the start tag and the end tag of an XML element
 - PCDATA WILL be parsed by a parser
 - Examined by the parser for entities and markup
 - Tags inside the text will be treated as markup and entities will be expanded
 - PCDATA should not contain any &, <, or > characters
 - These need to be represented by the & < and > entities, respectively
- CDATA (character data)
 - Text that will NOT be parsed by a parser
 - Tags inside the text will NOT be treated as markup and entities will not be expanded

3.3.5.2 DTD Elements

- Syntax
 - <!ELEMENT element-name category>, or
 - <!ELEMENT element-name (element-content)>
- Empty elements
 - Have category keyword EMPTY
 - Syntax: <!ELEMENT element-name EMPTY>
 - Example: <!ELEMENT br EMPTY>
- Elements with Parsed Character Data
 - Declared with #PCDATA inside parentheses

- Syntax: `<!ELEMENT element-name (#PCDATA)>`
- Example: `<!ELEMENT from (#PCDATA)>`
- Elements with any contents
 - Have category keyword ANY
 - Can contain any combination of parsable data
 - Syntax: `<!ELEMENT element-name ANY>`
 - Example: `<!ELEMENT note ANY>`
- Elements with Children (sequences)
 - Declared with the name of the children elements inside parentheses
 - Syntax: `<!ELEMENT element-name (child1)>` **or**
`<!ELEMENT element-name (child1,child2,...)>`
 - Example: `<!ELEMENT note (to,from,heading,body)>`
- When children are declared in a sequence separated by commas, the children must appear in the same sequence in the document
 - The full declaration of the "note" element

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

- Declaring Only One Occurrence of an Element
 - Syntax: `<!ELEMENT element-name (child-name)>`
 - Example: `<!ELEMENT note (message)>`
 - Child element "message" must occur once, and only once inside the "note" element
- Declaring Minimum One Occurrence of an Element
 - Syntax: `<!ELEMENT element-name (child-name+)>`
 - Example: `<!ELEMENT note (message+)>`
 - The child element "message" must occur one or more times inside the "note" element.
- Declaring Zero or More Occurrences of an Element
 - Syntax: `<!ELEMENT element-name (child-name*)>`
 - Example: `<!ELEMENT note (message*)>`

- The child element "message" can occur zero or more times inside the "note" element.
- Declaring Zero or One Occurrence of an Element
 - Syntax: `<!ELEMENT element-name (child-name?)>`
 - Example: `<!ELEMENT note (message?)>`
 - The child element "message" can occur zero or one time inside the "note" element.
- Declaring either/or Content
 - Example: `<!ELEMENT note (to,from,header,(message|body))>`
 - The "note" element must contain a "to" element, a "from" element, a "header" element, and either a "message" or a "body" element
- Declaring Mixed Content
 - Example: `<!ELEMENT note (#PCDATA|to|from|header|message)*>`
 - The "note" element can contain zero or more occurrences of parsed character data, "to", "from", "header", or "message" elements

3.3.5.3 DTD Attributes

- Declared with an ATTLIST declaration
 - Syntax: `<!ATTLIST element-name attribute-name attribute-type attribute-value>`
 - DTD Example: `<!ATTLIST payment type CDATA "check">`
 - Corresponding XML Example: `<payment type="check" />`
- Several attribute types: CDATA, ID, ENTITY, etc.
- Attribute value can be one of the following:

Value	Explanation
<i>value</i>	The default value of the attribute
#REQUIRED	The attribute is required
#IMPLIED	The attribute is optional
#FIXED <i>value</i>	The attribute value is fixed

See https://www.w3schools.com/xml/xml_dtd_attributes.asp for usage

3.3.6 XML Schema

- An XML Schema describes the structure of an XML document
- The XML Schema language is also referred to as XML Schema Definition (XSD)
- Below, an XML document followed by a possible schema (**note.xsd**)

```
<?xml version="1.0"?>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="https://www.w3schools.com"
xmlns="https://www.w3schools.com"
elementFormDefault="qualified">

  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

- The note element is a **complex type** because it contains other elements
- The other elements (to, from, heading, body) are **simple types** because they do not contain other elements
- **Validation**
- The XML document must refer to the schema that will be used to validate it

```
<?xml version="1.0"?>

<note
  xmlns="https://www.w3schools.com"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://www.w3schools.com/xml note.xsd">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

- **XSD <schema> element**

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="https://www.w3schools.com"
  xmlns="https://www.w3schools.com"
  elementFormDefault="qualified">
  ...
  ...
</xs:schema>
```

- **xmlns:xs="http://www.w3.org/2001/XMLSchema"** indicates that
 - The elements and data types used in the schema come from the "http://www.w3.org/2001/XMLSchema" namespace
 - The elements and data types that come from the "http://www.w3.org/2001/XMLSchema" namespace should be prefixed with **xs**:
- **targetNamespace="https://www.w3schools.com"** indicates that the elements defined by this schema (note, to, from, heading, body) come from the "https://www.w3schools.com" namespace
- **xmlns="https://www.w3schools.com"** indicates that the default namespace is "https://www.w3schools.com"
- **elementFormDefault="qualified"** indicates that any elements used by the XML instance document which were declared in this schema must be namespace qualified

- **XSD Elements**

- Can be simple or complex
 - Simple: Can contain only "text" (boolean, string, date, etc.)

- Custom (Complex): defined by you
- **SimplElements**
 - Syntax: `<xs:element name="xxx" type="yyy"/>`
 - Common types: `xs:string`, `xs:decimal`, `xs:integer`, `xs:boolean`, `xs:date`, `xs:time`
 - Example: an XML file and corresponding simple element definitions

```
<lastname>Refsnes</lastname>
<age>36</age>
<dateborn>1970-03-27</dateborn>
```

```
<xs:element name="lastname" type="xs:string"/>
<xs:element name="age" type="xs:integer"/>
<xs:element name="dateborn" type="xs:date"/>
```

- Simple elements may have a
 - default value (automatically assigned to the element when no other value is specified), or
 - fixed value (automatically assigned to the element, and you cannot specify another value)
 - Examples

```
<xs:element name="color" type="xs:string" default="red"/>
```

```
<xs:element name="color" type="xs:string" fixed="red"/>
```

▪ XSD Attributes

- All attributes are simple types
- Simple elements cannot have attributes
- Only complex elements can have attributes; the attributes themselves must be declared as a simple type
- Syntax for defining XSD attribute: `<xs:attribute name="xxx" type="yyy"/>`
- Example: XML with attribute, and corresponding attribute definition in XSD file

```
<lastname lang="EN">Smith</lastname>
```

```
<xs:attribute name="lang" type="xs:string"/>
```

- Like simple elements, attributes can have default or fixed values

```
<xs:attribute name="lang" type="xs:string" default="EN"/>
```

```
<xs:attribute name="lang" type="xs:string" fixed="EN"/>
```

- Attributes can also be optional or required.
- Attributes are optional by defaults

```
<xs:attribute name="lang" type="xs:string" use="required"/>
```

▪ Restrictions on values

- Example: minimum and maximum values for age
- Example: restriction to set of values

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

```
<xs:element name="car">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Audi"/>
      <xs:enumeration value="Golf"/>
      <xs:enumeration value="BMW"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

or equivalently,

```
<xs:element name="car" type="carType"/>

<xs:simpleType name="carType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Audi"/>
    <xs:enumeration value="Golf"/>
    <xs:enumeration value="BMW"/>
  </xs:restriction>
</xs:simpleType>
```

- Several other restrictions on values
 - See https://www.w3schools.com/xml/schema_facets.asp
- **Complex Elements**
 - Contains other elements and/or attributes
 - Four kinds of complex elements
 - empty elements
 - elements that contain only other elements
 - elements that contain only text
 - elements that contain both other elements and text

See https://www.w3schools.com/xml/schema_complex_elements.asp for further discussion of complex type.

XML Example Referencing a DTD

```
1  <?xml version = "1.0"?>
2
3  <!-- letter.xml -->
4  <!-- Business letter marked up with XML -->
5  <!DOCTYPE letter SYSTEM "letter.dtd">
6
7  <letter>
8      <contact type = "sender">
9          <name>Jane Doe</name>
10         <address1>Box 12345</address1>
11         <address2>15 Any Ave.</address2>
12         <city>Othertown</city>
13         <state>Otherstate</state>
14         <zip>67890</zip>
15         <phone>555-4321</phone>
16         <flag gender = "F" />
17     </contact>
18
19     <contact type = "receiver">
20         <name>John Doe</name>
21         <address1>123 Main St.</address1>
22         <address2></address2>
23         <city>Anytown</city>
24         <state>Anystate</state>
25         <zip>12345</zip>
26         <phone>555-1234</phone>
27         <flag gender = "M" />
28     </contact>
29
30     <salutation>Dear Sir:</salutation>
31
32     <paragraph>It is our privilege to inform you about our new database
33         managed with XML. This new system allows you to reduce the
34         load on your inventory list server by having the client machine
35         perform the work of sorting and filtering the data.
36     </paragraph>
37
38     <paragraph>Please visit our website for availability and pricing.
39     </paragraph>
40
41     <closing>Sincerely,</closing>
42     <signature>Ms. Jane Doe</signature>
43 </letter>
```

DOCTYPE reference (Line 5)

- Contains three items
 - The name of the root element that the DTD specifies (letter)
 - Keyword SYSTEM denoting an external DTD (a DTD declared in a separate file, as opposed to a DTD declared locally in the same file)
 - DTD's name and location (i.e., letter.dtd in the current directory)

The XML Document's Contents

- Root element: letter
- Children of root element: contact, contact, salutation, paragraph, paragraph, closing and signature
- 2 ways to place data
 - Between an element's tags
 - As attributes, i.e., name/value pairs that appear within the angle brackets of an element's start tag
 - (Elements can have any number of attributes (separated by spaces) in their start tags)
- First contact element (lines 8–17) has an attribute named **type** with attribute value "**sender**"
- Second contact element (lines 19–28) has attribute **type** with value "**receiver**"
- A contact element stores various items of information about a contact, such as the contact's name (element name), address (elements address1, address2, city, state and zip), phone number (element phone) and gender (represented by attribute gender of element flag)
- Element salutation (line 30) marks up the letter's salutation
- Lines 32–39 mark up the letter's body using two paragraph elements
- Elements closing (line 41) and signature (line 42) mark up the closing sentence and the author's "signature," respectively
- Empty elements
 - Do not have any content
 - Instead, it sometimes places data in attributes
- Two ways to handle empty elements
 - E.g., Line 16
 - Empty element **flag**
 - Has one attribute that indicates the gender of the contact element
 - This empty element is closed by placing a slash immediately preceding the right angle bracket
 - Another way to handle empty elements
 - Explicitly write an end tag (e.g., Line 22): <address2></address2>
 - Note
 - We must include this element to conform to the structural rules specified in the XML document's DTD (see later), which states that each contact element must have an address2 child element (even if it is empty)
 - Otherwise, we don't have to include it

Document Type Definition

```
1  <!-- letter.dtd -->
2  <!-- DTD document for letter.xml -->
3
4  <!ELEMENT letter ( contact+, salutation, paragraph+,
5  closing, signature )>
6
7  <!ELEMENT contact ( name, address1, address2, city, state,
8  zip, phone, flag )>
9  <!ATTLIST contact type CDATA #IMPLIED>
10
11 <!ELEMENT name ( #PCDATA )>
12 <!ELEMENT address1 ( #PCDATA )>
13 <!ELEMENT address2 ( #PCDATA )>
14 <!ELEMENT city ( #PCDATA )>
15 <!ELEMENT state ( #PCDATA )>
16 <!ELEMENT zip ( #PCDATA )>
17 <!ELEMENT phone ( #PCDATA )>
18 <!ELEMENT flag EMPTY>
19 <!ATTLIST flag gender (M | F) "M">
20
21 <!ELEMENT salutation ( #PCDATA )>
22 <!ELEMENT closing ( #PCDATA )>
23 <!ELEMENT paragraph ( #PCDATA )>
24 <!ELEMENT signature ( #PCDATA )>
```

A DTD expresses the set of rules for document structure using an EBNF (Extended Backus-Naur Form) grammar

- EBNF grammars are commonly used to define programming languages

Defining Elements in a DTD

- **ELEMENT *element type*** declaration is used to define the rules for an element
- Example
 - Lines 4–5 defines the rules for element **letter**
 - Letter contains one or more **contact** elements, one **salutation** element, one or more **paragraph** elements, one **closing** element and one **signature** element, in that sequence
 - Plus sign (+) occurrence indicator specifies that the DTD requires one or more occurrences of an element
 - Other occurrence indicators
 - Asterisk (*), which indicates an optional element that can occur zero or more times
 - Question mark (?), which indicates an optional element that can occur at most once (i.e., zero or one occurrence)
 - Absence of an occurrence indicator means that the DTD requires exactly one occurrence

Defining Attributes in a DTD

- **ATTLIST *attribute-list*** declaration is used to define attributes of an element
- Example
 - Line 9 defines an attribute named **type** for the contact element
 - Keyword **#IMPLIED** specifies that if the parser finds a contact element without a **type** attribute, the parser can choose an arbitrary value for the attribute or can ignore the attribute
 - Other possible keywords in an ATTLIST declaration include
 - **#REQUIRED**: specifies that the attribute must be present in the element
 - **#FIXED**: specifies that the attribute (if present) must have the given fixed value
 - Example
 - `<!ATTLIST address zip CDATA #FIXED "01757">` indicates that attribute zip (if present in element address) must have the value 01757

Character Data vs. Parsed Character Data

- Keyword **CDATA** (e.g., Line 9)
 - Specifies that the attribute (**type** in this example) contains character data (i.e., a string)
 - A parser will pass such data to an application without modification
- Keyword **PCDATA** (e.g., Line 11)
 - Specifies that an element (**name** in this example) may contain parsed character data
 - i.e., data that's processed by an XML parser
 - Elements with parsed character data cannot contain markup characters, such as `<`, `>` or `&`
 - Any markup character in a **PCDATA** must be replaced with the character's corresponding character entity reference
 - Examples
 - Character entity reference **<** should be used in place of the less-than symbol (`<`)
 - Character entity reference **>** should be used in place of the greater-than symbol (`>`)
 - Note
 - To use a literal ampersand, the corresponding entity reference **&** should be used instead
 - Parsed character data can contain ampersands (`&`) only for inserting entities

Defining Empty Elements in a DTD

- Keyword **EMPTY** specifies that the element does not contain any data between its start and end tags
 - Example, Line 18 defines an empty element named **flag**
- Empty elements commonly describe data via attributes
 - Example, **flag's** data appears in its **gender** attribute (Line 19)
 - In this example
 - The **gender** attribute's value must be one of the enumerated values (M or F), and
 - **gender** has a default value of M

Validating Against an XML Schema Document

```
1  <?xml version = "1.0"?>
2
3  <!-- book.xml -->
4  <!-- Book list marked up as XML -->
5  <deitel:books xmlns:deitel = "http://www.deitel.com/booklist">
6    <book>
7      <title>Visual Basic 2010 How to Program</title>
8    </book>
9    <book>
10     <title>Visual C# 2010 How to Program, 4/e</title>
11   </book>
12   <book>
13     <title>Java How to Program, 9/e</title>
14   </book>
15   <book>
16     <title>C++ How to Program, 8/e</title>
17   </book>
18   <book>
19     <title>Internet and World Wide Web How to Program, 5/e</title>
20   </book>
21 </deitel:books>
```

```
1  <?xml version = "1.0"?>
2
3  <!-- Fig. 15.10: book.xsd -->
4  <!-- Simple W3C XML Schema document -->
5  <schema xmlns = "http://www.w3.org/2001/XMLSchema"
6    xmlns:deitel = "http://www.deitel.com/booklist"
7    targetNamespace = "http://www.deitel.com/booklist">
8
9    <element name = "books" type = "deitel:BooksType"/>
10
11    <complexType name = "BooksType">
12      <sequence>
13        <element name = "book" type = "deitel:SingleBookType"
14          minOccurs = "1" maxOccurs = "unbounded"/>
15      </sequence>
16    </complexType>
17
18    <complexType name = "SingleBookType">
19      <sequence>
20        <element name = "title" type = "string"/>
21      </sequence>
22    </complexType>
23  </schema>
```

Top figure: A schema-valid XML document named book.xml

Bottom figure: XML Schema document (book.xsd) that defines the structure for book.xml

Creating an XML Schema Document

- book.xsd defines an XML-based language (i.e., a vocabulary) for writing XML documents about collections of books
 - defines the elements, attributes and parent/child relationships that such a document can (or must) include, and specifies the type of data that these elements and attributes may contain
- Root element: schema
 - Contains elements that define the structure of an XML document
 - Line 5: specifies as the default namespace, the standard W3C XML Schema namespace URI – <http://www.w3.org/2001/XMLSchema>
 - This namespace contains predefined elements that comprise the XML Schema vocabulary (i.e., the language used to write an XML Schema document)
- Line 6
 - Binds the URI <http://www.deitel.com/booklist> to namespace prefix **deitel**
 - The schema uses this namespace to differentiate names created by us (deitel) from names that are part of the XML Schema namespace
- Line 7
 - Also specifies <http://www.deitel.com/booklist> as the **targetNamespace** of the schema
 - This attribute identifies the namespace of the XML vocabulary that this schema defines
 - Note: this is same as the namespace referenced in line 5 of book.xml
 - This is what “connects” the XML document with the schema that defines its structure

Defining an Element in XML Schema

- The element tag (e.g., Line 9)
 - defines an element to be included in an XML document that conforms to the schema
 - i.e., the actual elements that can be used to mark up data
 - **books** in this example, which we use as the root element in book.xml
 - Attributes **name** and **type** specify the element’s name and type, respectively
 - An element’s type indicates the data that the element may contain
 - Possible types include
 - XML Schema-defined types (e.g., string, double) and
 - User-defined types (e.g., BooksType, Lines 11–16)

- Example, Element **books** (Line 9)
 - Type of element **books** is **deitel:BooksType**
 - BooksType (Lines 11 - 16) is a user-defined type in the namespace <http://www.deitel.com/booklist> and therefore must have the namespace prefix **deitel**
 - Two categories of type exist in XML Schema
 - Simple types: cannot contain attributes or child elements
 - Complex types can contain attributes and/or child elements
 - A user-defined type that contains attributes or child elements must be defined as a complex type
 - Lines 11–16 use element **complexType** to define BooksType as a complex type that has a child element named book
 - Element **sequence** (Lines 12–15) allows you to specify the sequential order in which child elements must appear (in this example, there is only one element – book – in the sequence)
 - Lines 13–14 indicates that a BooksType element (books in this example) can contain child elements named book of type deitel:SingleBookType (defined in lines 18–22)
 - Attribute minOccurs in this example specifies that elements of type BooksType must contain a minimum of one book element
 - Attribute maxOccurs in this example specifies that elements of type BooksType may have any number of book child elements
 - SingleBookType (Lines 18–22)
 - define a complex type
 - An element of this type contains a child element named **title**
 - Element **title** (Line 20) is defined to be of simple type string

More on Types in XML Schema

- Every element in XML Schema has a type, which may be the built-in or user-defined
- Simple types
 - Every simple type defines a restriction on an XML Schema-defined type or a restriction on a user-defined type
 - Restrictions limit the possible values that an element can hold
- Complex types
 - Divided into two groups: those with simple content and those with complex content

- Both can contain attributes, but only complex content can contain child elements
 - Complex types with simple content must extend or restrict some other existing type
 - Complex types with complex content do not have this limitation

```

1  <?xml version = "1.0"?>
2  <!-- computer.xsd -->
3  <!-- W3C XML Schema document -->
4
5  <schema xmlns = "http://www.w3.org/2001/XMLSchema"
6          xmlns:computer = "http://www.deitel.com/computer"
7          targetNamespace = "http://www.deitel.com/computer">
8
9      <simpleType name = "gigahertz">
10         <restriction base = "decimal">
11             <minInclusive value = "2.1"/>
12         </restriction>
13     </simpleType>
14
15     <complexType name = "CPU">
16         <simpleContent>
17             <extension base = "string">
18                 <attribute name = "model" type = "string"/>
19             </extension>
20         </simpleContent>
21     </complexType>
22
23     <complexType name = "portable">
24         <all>
25             <element name = "processor" type = "computer:CPU"/>
26             <element name = "monitor" type = "int"/>
27             <element name = "CPUSpeed" type = "computer:gigahertz"/>
28             <element name = "RAM" type = "int"/>
29         </all>
30         <attribute name = "manufacturer" type = "string"/>
31     </complexType>
32
33     <element name = "laptop" type = "computer:portable"/>
34 </schema>

```

```

1  <?xml version = "1.0"?>
2
3  <!-- laptop.xml -->
4  <!-- Laptop components marked up as XML -->
5  <computer:laptop xmlns:computer = "http://www.deitel.com/computer"
6                  manufacturer = "IBM">
7
8      <processor model = "Centrino">Intel</processor>
9      <monitor>17</monitor>
10     <CPUSpeed>2.4</CPUSpeed>
11     <RAM>256</RAM>
12 </computer:laptop>

```

Example above (computer.xsd) that describes parts of a laptop computer, demonstrates complex types with each kind of content, followed by an XML document (laptop.xml) that follows the structure defined in computer.xsd

- Line 5
 - Declares the default namespace to be the standard XML Schema namespace
 - Any elements without a prefix are assumed to be in that namespace
- Line 6
 - Binds the namespace prefix **computer** to the namespace <http://www.deitel.com/computer>
 - Line 7 identifies this namespace as the targetNamespace

XML elements for describing laptop computers

- **simpleType** element gigahertz (Lines 9-13)
 - Used here to describe the processor clock speed
 - Simple types are restrictions of a type typically called a **base type** (*decimal* in this example), with value restricted to be at least 2.1 by using the **minInclusive** element
- **complexType** CPU
 - Has **simpleContent**
 - i.e., can have attributes but not child elements, and must extend or restrict some XML Schema type or user-defined type
 - Element **extension** with **attribute** base (Line 17) sets the base type to string
 - Element **attribute** (Line 18) gives the complexType an attribute named **model** of type string
 - Hence, an element of type CPU must contain string text (the base type is string) and may contain a **model** attribute that's also of type string
- **complexType** portable (Lines 23–31)
 - Can have child elements and attributes
 - Element **all**
 - Encloses elements that must each be included (in any order) once in the corresponding XML instance document
 - In this example, four elements – processor, monitor, CPUSpeed with types CPU, int, gigahertz and int, respectively
 - Note: Must include namespace prefix computer for types CPU and gigahertz, because these user-defined types are part of the computer namespace (<http://www.deitel.com/computer>)

- Also, portable contains an attribute (element ***attribute*** – Line 30) called manufacturer of type string
- Line 33
 - Declares the actual element that uses the types defined in the schema
 - Element name is laptop and is of type portable, i.e., contains child elements processor, monitor, CPUSpeed and RAM, and an attribute manufacturer

The XML file

- Line 5 declares namespace prefix ***computer***, required by the laptop element because it's part of the `http://www.deitel.com/computer` namespace
- Line 6: the laptop's manufacturer attribute
- Lines 8–11: the elements defined in the schema to describe the laptop's characteristics